

```

subHrEmp.setHourlyRate( 5.67 );
subHrEmp.setHoursWorked( 20 );
pay = subHrEmp. calcPay();
System.out.println( "Subclass pay = " + pay );
}
public static void main( String[] args )
{
    CastSubTypeReftoSuperTypeObject demo;
    demo = new CastSubTypeReftoSuperTypeObject();
    System.exit( 0 );
}

```

Exception in thread "main" java.lang.ClassCastException: Employee at astSubTypeReftoSuperTypeObject.<init>(CastSubTypeReftoSuperTypeObject.java:60) at astSubTypeReftoSuperTypeObject.main(CastSubTypeReftoSuperTypeObject.java:75)

Press any key to continue...

At this stage, the compiler is incensed, it trusted us when we asked for that Explicit cast, and look what happened.

5.6 Polymorphism and dynamic binding

Say that we have a bunch of objects that are all Shapes: Circles, Squares, Triangles, Trapezoids, etc. If we want to draw them all, we could use a large switch statement that asks if the object is a circle, do this. If a square, do this. If a triangle, do that. The big switch could work okay, but if we decided to add a parallelogram to our shapes, then we would have to remember to add another branch to our switch statement. Every place we wanted to do some behavior based on the shape, we would have to change our logic.

Polymorphism can save us that trouble and make programs that will work with Shapes that hadn't even been thought of when the class was written. With Polymorphism, the objects themselves know what kind of object they are, and they do the right behavior based on their internal awareness of their own type. In our Shapes example, we could assume that each shape Subclass that inherits off of the master Superclass—Shape—would have a draw() method. That draw() method would detail exactly how to generate the Square or the Circle or the Triangle. Each shape subclass knows how to draw itself. Certainly, we could go through the trouble of drawing each Shape one by one. But wouldn't it be nice if we could just treat all those various Shape Subclasses as copies of the Superclass? Then, we could create an array of Superclass references that